



实验二 模型机组合部件的实现（一）（实验报告格式案例）

班级 _____姓名 学号

一、实验目的

1. 了解简易模型机的内部结构和工作原理。
2. 熟悉译码器、运算器的工作原理。
3. 分析模型机的功能，设计指令译码器。
4. 分析模型机的功能，设计 ALU。

二、实验内容

1. 用 VERILOG 语言设计指令译码器；
2. 用 VERILOG 语言设计 ALU。

三、实验过程

1、指令译码器

A）创建工程（选择的芯片为 family=Cyclone II；name=EP2C5T144C8）
步骤：左上角 file->New Project Wizard->选择工程位置和工程名->选择芯片 Cyclone II，available device 中选择 EP2C5T144C8->点击 next->最后点击 finish 完成创建工程
工程创建图：



PDF-XChange Editor Click to BUY NOW! www.tracker-software.com

B) 编写源代码

根据实验指导和要求实现的功能写出对应的 Verilog 代码。

步骤：左上角 file->new->Verilog hdl file->编写代码（模块名需与工程名一致）->编译成功后保存到工程文件中

代码图：

```
1  module yimaqil(en,ir,mova,movb,movc,add,sub,andl,notl,rsr,rsl,jmp,jz,jc,inl,outl,nop,halt);
2    input wire en;
3    input wire [7:0] ir;
4    output reg mova,movb,movc,add,sub,andl,notl,rsr,rsl,jmp,jz,jc,inl,outl,nop,halt;
5    always @(ir)
6      begin {mova,movb,movc,add,sub,andl,notl,rsr,rsl,jmp,jz,jc,inl,outl,nop,halt}=0;
7      if(en==1'b1)
8        begin
9        if(ir[7:4]==4'b1100) begin
10           if(ir[3:2]==2'b11) movb=1;
11           else if(ir[1:0]==2'b11) movc=1;
12           else mova=1;
13       end
14       else if(ir[7:4]==4'b1001) add=1;
15       else if(ir[7:4]==4'b0110) sub=1;
16       else if(ir[7:4]==4'b1011) andl=1;
17       else if(ir[7:4]==4'b0101) notl=1;
18       else if(ir[7:4]==4'b1010) begin
19              if(ir[1:0]==2'b00) rsr=1;
20              else if(ir[1:0]==2'b11) rsl=1;
21       end
22       else if(ir[7:2]==6'b001100) begin
23              if(ir[1:0]==2'b00) jmp=1;
24              else if(ir[1:0]==2'b01) jz=1;
25              else if(ir[1:0]==2'b10) jc=1;
26       end
27       else if(ir[7:4]==4'b0010) inl=1;
28       else if(ir[7:4]==4'b0100) outl=1;
29       else if(ir[7:0]==8'b01110000) nop=1;
30       else if(ir[7:0]==8'b10000000) halt=1;
31       end
32 end
33
34 endmodule
35
36
```

C) 编译与调试(包含编译调试过程中的错误、警告信息以及资源消耗)

确定源代码文件为当前工程文件，点击【processing】-【start compilation】进行文件编译，编译成功，保存文件。

错误：

```
Info: Command: quartus_map --read_settings_files=on --write_settings_files=off yimaqil -c yimaqil
Error (10170): Verilog HDL syntax error at yimaqil.v(14) near text "else";  expecting ";", or "@", or "end", or an identifier ("else" is a reserved keyword ), or a sy
Error (10112): Ignored design unit "yimaqil" at yimaqil.v(1) due to previous errors
Info: Found 0 design units, including 0 entities, in source file yimaqil.v
Error: Quartus II Analysis & Synthesis was unsuccessful. 2 errors, 0 warnings
Error: Quartus II Full Compilation was unsuccessful. 4 errors, 0 warnings
```

错误原因：

```
10          if(ir[3:2]==2'b11) movb=1;
11          else if(ir[1:0]==2'b11) movc=1;
12          else mova=1;
13      end;
14      else if(ir[7:4]==4'b1001) add=1;
15      else if(ir[7:4]==4'b0110) sub=1;
16      else if(ir[7:4]==4'b1011) andl=1;
17      else if(ir[7:4]==4'b0101) notl=1;
```

修改：end 后不需要接分号，将分号删除即可。

警告信息：

```
Warning (10235): Verilog HDL Always Construct warning at yimaqi.v(7): variable "en" is read inside the Always Construct but isn't in the Always Construct's Event Contro
Warning: Feature LogicLock is not available with your current license
Warning: No exact pin location assignment(s) for 25 pins of 25 total pins
Warning: Found 16 output pins without output pin load capacitance assignment
Warning: The Reserve All Unused Pins setting has not been specified, and will default to 'As output driving ground'.
```

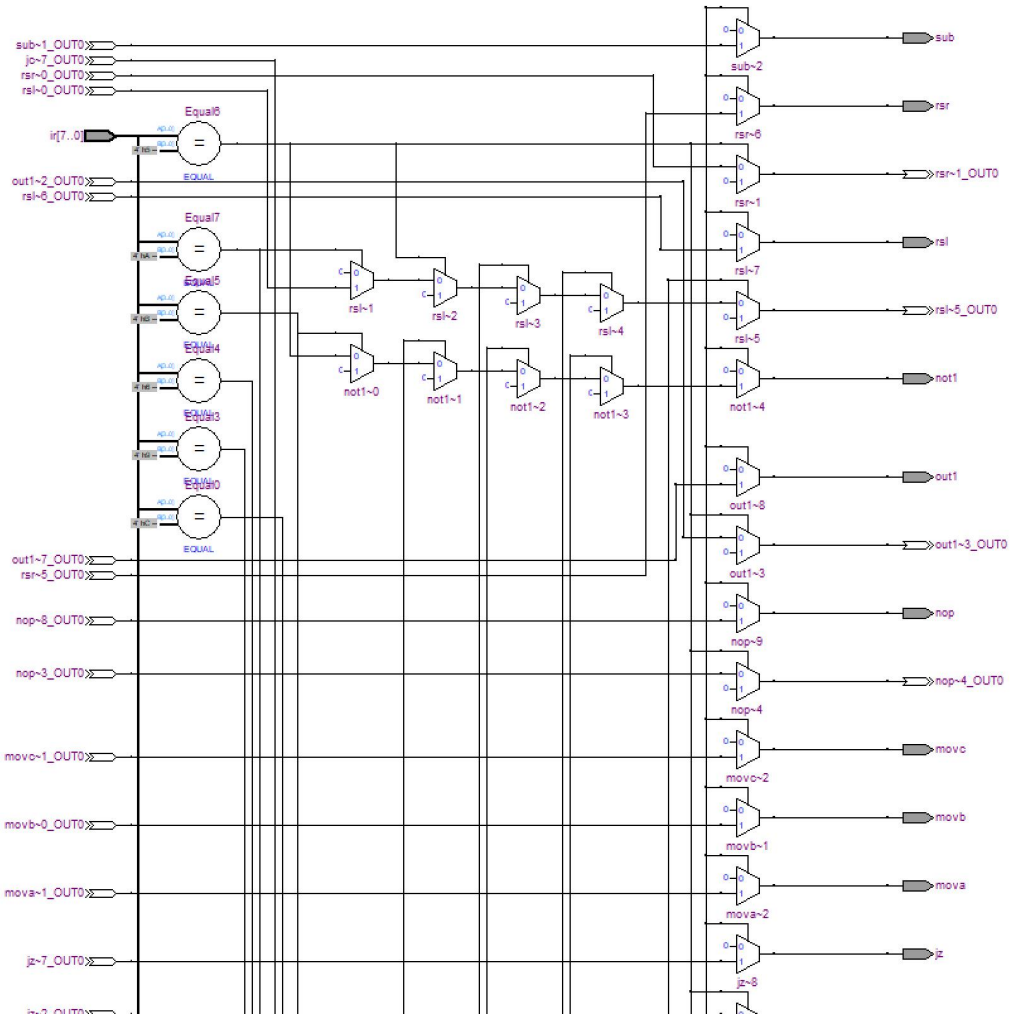



资源消耗：

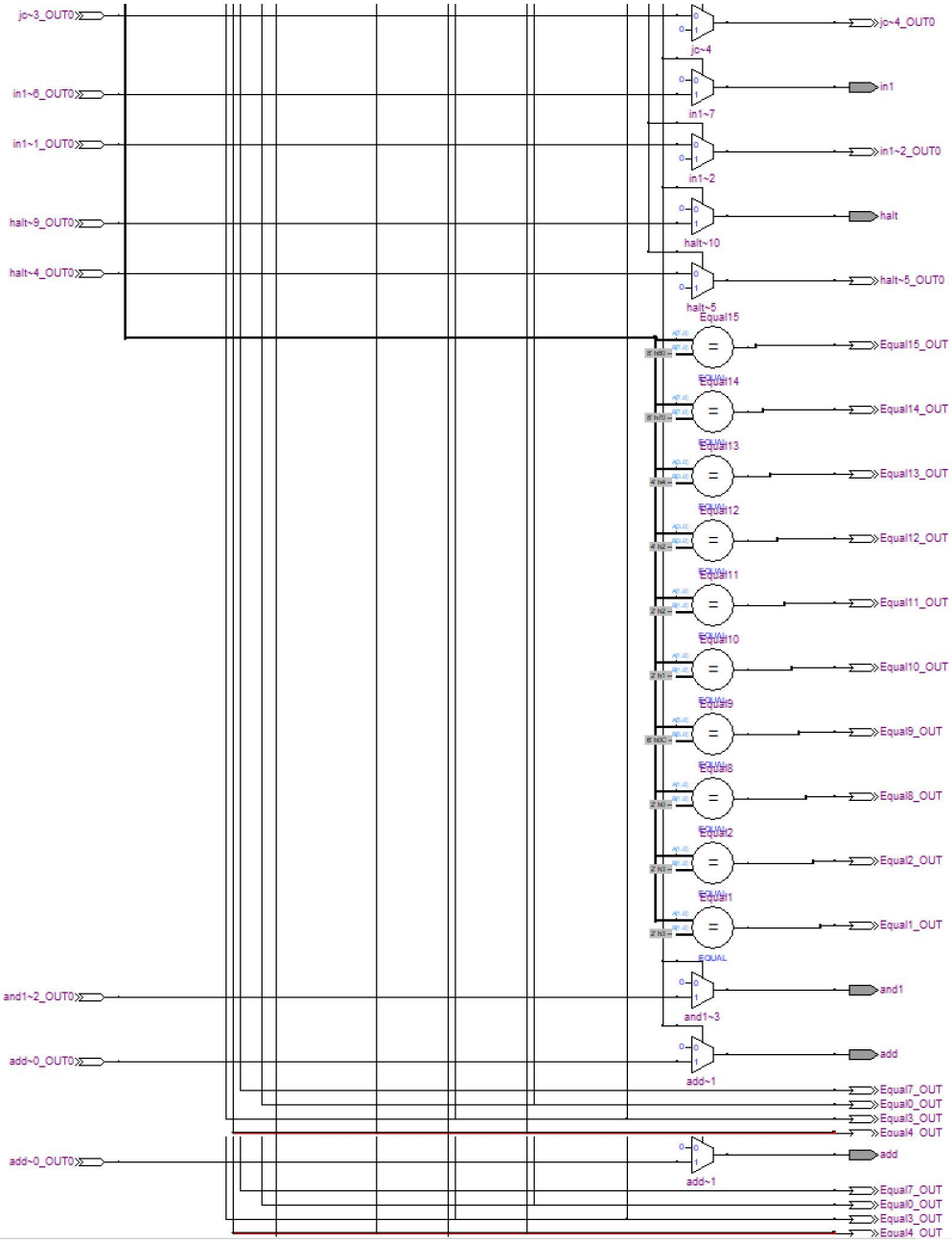
Flow Summary

Flow Status	Successful - Mon Dec 06 20:26:46 2021
Quartus II Version	9.0 Build 235 06/17/2009 SP 2 SJ Web Edition
Revision Name	yimaqi1
Top-level Entity Name	yimaqi1
Family	Cyclone II
Device	EP2C5T144C8
Timing Models	Final
Met timing requirements	Yes
Total logic elements	30 / 4,608 (< 1 %)
Total combinational functions	30 / 4,608 (< 1 %)
Dedicated logic registers	0 / 4,608 (0 %)
Total registers	0
Total pins	25 / 89 (28 %)
Total virtual pins	0
Total memory bits	0 / 119,808 (0 %)
Embedded Multiplier 9-bit elements	0 / 26 (0 %)
Total PLLs	0 / 2 (0 %)

D) RTL 视图



PDF-XChange Editor
Click to BUY NOW!
www.tracker-software.com



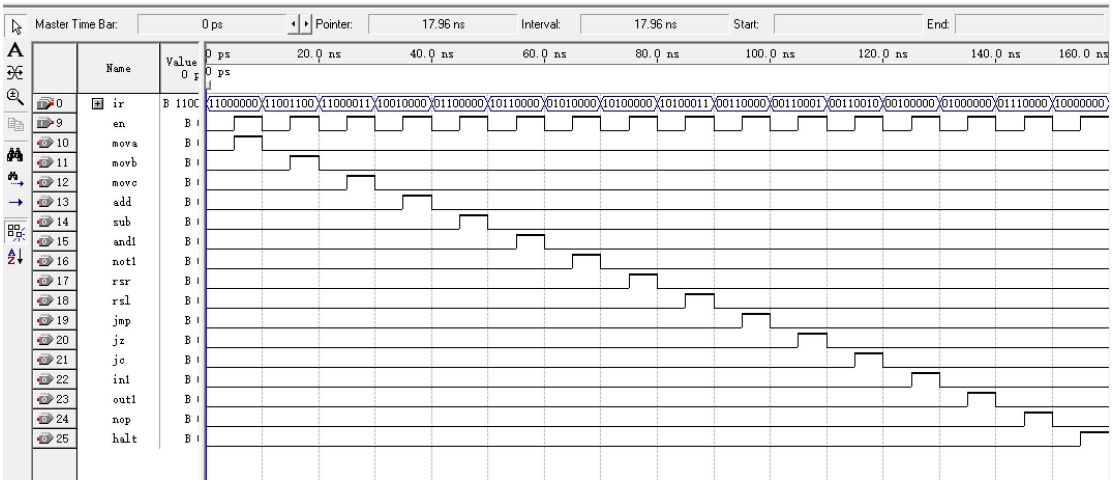
视图分析及结论：

分析：由视图可得，视图左边为输入，右边为输出。其中连接有一系列的元器件。比如比较器：当输入相等时输出 1，不相等时输出 0；还有大部分的 2-1 选择器构成，当控制信号为 0 时，输出第一位，控制信号为 1 时，输出第二位。图中输入信号为 ir 和 en，输出信号包括 add 等 16 种情况。各个输出端口之间通过导线相连。

结论：一个功能的实现需要经过多重门的处理后才能实现，一个元件的内部原理结构图十分复杂。

PDF-XChange Editor Click to BUY NOW! www.tracker-software.com

E) 功能仿真波形



结果分析及结论：

分析：功能仿真是指不考虑器件延时和布线延时的理想情况下对源代码进行逻辑功能的验证。由仿真波形可得，对于输入状态的变化，输出结果实时变化，没有延迟，其结果与电路设计的真值表的结果相对应。

当 en 为 0 时，不管 ir 为何值，16 个输出全为 0

当 en 为 1 时：

当 ir=11000000 时，mova 输出为 1；

当 ir=11001100 时，movb 输出为 1；

当 ir=11000011 时，movc 输出为 1；

当 ir=10010000 时，add 输出为 1；

当 ir=01100000 时，sub 输出为 1；

当 ir=10110000 时，and1 输出为 1；

当 ir=01010000 时，not1 输出为 1；

当 ir=10100000 时，rsr 输出为 1；

当 ir=10100011 时，rsl 输出为 1；

当 ir=00110000 时，jmp 输出为 1；

当 ir=00110001 时，jz 输出为 1；

当 ir=00110010 时，jc 输出为 1；

当 ir=00100000 时，in1 输出为 1；

当 ir=01000000 时，out1 输出为 1；

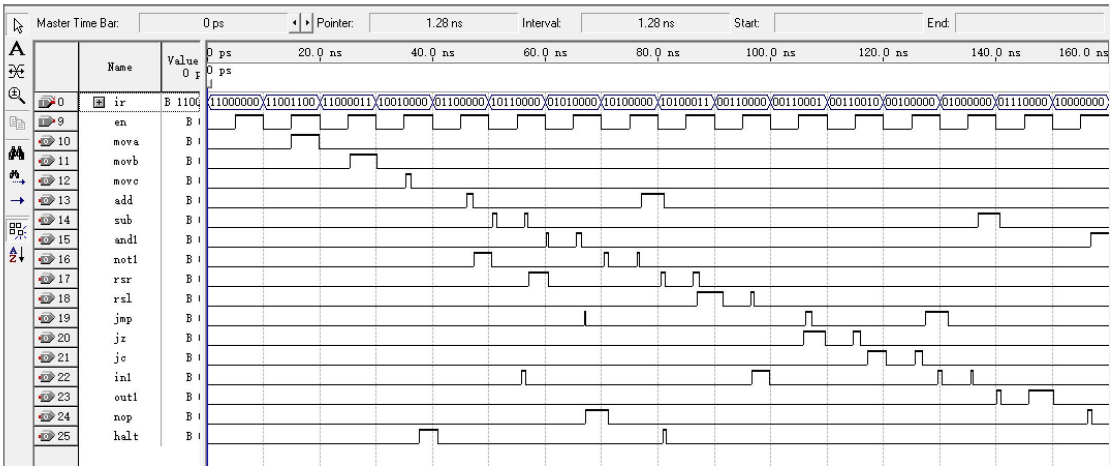
当 ir=01110000 时，nop 输出为 1；

当 ir=10000000 时，halt 输出为 1；

结论：功能仿真操作简单，能体现和验证实验的功能，但忽略延迟的影响会使结果与实际结果有一定误差。

PDF-XChange Editor Click to BUY NOW! www.tracker-software.com

F) 时序仿真波形



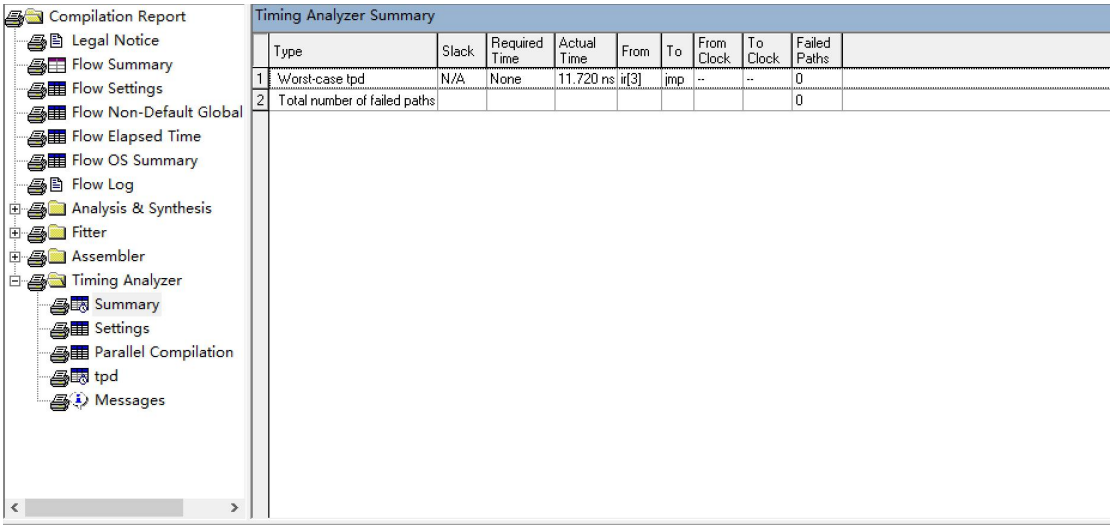
结果分析及结论：

分析：时序仿真是指在布线后进行，是最接近真实器件运行的仿真，它与特定的器件有关，又包含了器件和布线的延时信息。由波形可得，当输入状态发生改变时，输出结果并未同时改变，而是有一定延迟，同时由于输入状态的改变，导致电路出现“冒险”，导致输出结果并未与预期结果相同。

结论：时序仿真可以用来验证程序在目标器件中的时序关系。同时考虑了器件的延迟后，其输出结果跟接近实际情况，但是考虑的情况过多，不容易操作，容易产生错误。时序仿真不仅反应出输出和输入的逻辑关系，同时还计算了时间的延时信息，是与实际系统更接近的一种仿真结果。不过，要注意的是，这个时间延时是仿真软件“估算”出来的。

G) 时序分析

操作方法是：编译后，在 compilation report 中选择【timing analysis】-【summary】-【tpd】



Timing Analyzer Summmary 图

PDF-XChange Editor Click to BUY NOW! www.tracker-software.com

tpd

	Slack	Required P2P Time	Actual P2P Time	From	To
1	N/A	None	11.720 ns	ir[3]	jmp
2	N/A	None	11.699 ns	ir[5]	add
3	N/A	None	11.696 ns	ir[3]	nop
4	N/A	None	11.680 ns	ir[2]	jmp
5	N/A	None	11.656 ns	ir[2]	nop
6	N/A	None	11.639 ns	ir[3]	halt
7	N/A	None	11.595 ns	ir[2]	halt
8	N/A	None	11.512 ns	ir[3]	jc
9	N/A	None	11.497 ns	ir[5]	jmp
10	N/A	None	11.473 ns	ir[5]	nop
11	N/A	None	11.472 ns	ir[2]	jc
12	N/A	None	11.472 ns	ir[4]	jmp
13	N/A	None	11.466 ns	en	rsl
14	N/A	None	11.449 ns	en	halt
15	N/A	None	11.448 ns	ir[4]	nop
16	N/A	None	11.441 ns	en	sub
17	N/A	None	11.438 ns	ir[4]	add
18	N/A	None	11.384 ns	ir[5]	halt
19	N/A	None	11.308 ns	ir[5]	rsl
20	N/A	None	11.289 ns	ir[5]	jc
21	N/A	None	11.264 ns	ir[4]	jc
22	N/A	None	11.262 ns	en	rsr
23	N/A	None	11.253 ns	en	not1
24	N/A	None	11.224 ns	en	jmp
25	N/A	None	11.216 ns	ir[4]	rsl
26	N/A	None	11.206 ns	en	nop
27	N/A	None	11.028 ns	en	add
28	N/A	None	10.979 ns	ir[4]	halt
29	N/A	None	10.873 ns	en	out1
30	N/A	None	10.734 ns	ir[4]	sub
31	N/A	None	10.731 ns	ir[5]	sub
29	N/A	None	10.873 ns	en	out1
30	N/A	None	10.734 ns	ir[4]	sub
31	N/A	None	10.731 ns	ir[5]	sub
32	N/A	None	10.696 ns	ir[3]	movc
33	N/A	None	10.653 ns	ir[2]	movc
34	N/A	None	10.594 ns	ir[5]	movc
35	N/A	None	10.576 ns	ir[4]	not1
36	N/A	None	10.557 ns	en	jc
37	N/A	None	10.548 ns	ir[5]	not1
38	N/A	None	10.530 ns	ir[4]	rsr
39	N/A	None	10.526 ns	ir[5]	rsr
40	N/A	None	10.502 ns	en	and1
41	N/A	None	10.467 ns	en	in1
42	N/A	None	10.429 ns	ir[2]	movb
43	N/A	None	10.389 ns	ir[4]	movb
41	N/A	None	10.467 ns	en	in1
42	N/A	None	10.429 ns	ir[2]	movb
43	N/A	None	10.389 ns	ir[4]	movb
44	N/A	None	10.388 ns	en	movb
45	N/A	None	10.366 ns	ir[3]	jz
46	N/A	None	10.361 ns	ir[5]	movb
47	N/A	None	10.344 ns	ir[5]	and1
48	N/A	None	10.334 ns	en	movc
49	N/A	None	10.326 ns	ir[2]	jz
50	N/A	None	10.324 ns	ir[4]	movc
51	N/A	None	10.284 ns	ir[3]	mova
52	N/A	None	10.241 ns	ir[2]	mova
53	N/A	None	10.232 ns	ir[3]	movb
54	N/A	None	10.171 ns	ir[4]	out1
55	N/A	None	10.169 ns	ir[5]	out1
56	N/A	None	10.151 ns	ir[4]	and1
57	N/A	None	10.143 ns	ir[5]	jz
58	N/A	None	10.118 ns	ir[4]	jz
59	N/A	None	9.863 ns	en	mova
60	N/A	None	9.742 ns	ir[4]	in1
61	N/A	None	9.733 ns	ir[5]	in1
62	N/A	None	9.732 ns	ir[5]	mova
63	N/A	None	9.724 ns	en	jz
64	N/A	None	9.462 ns	ir[4]	mova
65	N/A	None	7.713 ns	ir[6]	rsl
66	N/A	None	7.657 ns	ir[1]	halt
67	N/A	None	7.606 ns	ir[7]	sub
68	N/A	None	7.530 ns	ir[0]	halt
69	N/A	None	7.477 ns	ir[7]	rsl
70	N/A	None	7.470 ns	ir[1]	rsr
71	N/A	None	7.432 ns	ir[1]	jmp
72	N/A	None	7.418 ns	ir[7]	not1
73	N/A	None	7.414 ns	ir[1]	nop
74	N/A	None	7.343 ns	ir[0]	rsr
75	N/A	None	7.339 ns	ir[7]	halt
76	N/A	None	7.305 ns	ir[0]	jmp
77	N/A	None	7.287 ns	ir[0]	nop
78	N/A	None	7.286 ns	ir[6]	jc
79	N/A	None	7.269 ns	ir[6]	halt
80	N/A	None	7.256 ns	ir[0]	jc
81	N/A	None	7.182 ns	ir[7]	add
82	N/A	None	7.147 ns	ir[7]	rsr
83	N/A	None	7.132 ns	ir[7]	jc
84	N/A	None	7.111 ns	ir[6]	add
85	N/A	None	7.109 ns	ir[6]	jmp
86	N/A	None	7.100 ns	ir[6]	nop
87	N/A	None	7.077 ns	ir[6]	rsr
88	N/A	None	7.038 ns	ir[7]	out1
89	N/A	None	7.035 ns	ir[0]	rsl
90	N/A	None	6.984 ns	ir[1]	jc
91	N/A	None	6.959 ns	ir[1]	rsl
92	N/A	None	6.954 ns	ir[7]	jmp
90	N/A	None	6.984 ns	ir[1]	jc
91	N/A	None	6.959 ns	ir[1]	rsl
92	N/A	None	6.954 ns	ir[7]	jmp
93	N/A	None	6.929 ns	ir[7]	nop
94	N/A	None	6.832 ns	ir[6]	sub
95	N/A	None	6.749 ns	ir[6]	and1
96	N/A	None	6.664 ns	ir[6]	not1
97	N/A	None	6.632 ns	ir[7]	in1
98	N/A	None	6.613 ns	ir[6]	movc
99	N/A	None	6.612 ns	ir[6]	jz
100	N/A	None	6.513 ns	ir[7]	and1
101	N/A	None	6.503 ns	ir[6]	movb
102	N/A	None	6.497 ns	ir[1]	jz
103	N/A	None	6.473 ns	ir[1]	movc
103	N/A	None	6.473 ns	ir[1]	movc
104	N/A	None	6.389 ns	ir[7]	movb
105	N/A	None	6.370 ns	ir[7]	jz
106	N/A	None	6.354 ns	ir[7]	movc
107	N/A	None	6.270 ns	ir[6]	out1
108	N/A	None	6.146 ns	ir[0]	movc
109	N/A	None	6.115 ns	ir[1]	mova
110	N/A	None	5.944 ns	ir[0]	mova
111	N/A	None	5.853 ns	ir[7]	mova
112	N/A	None	5.834 ns	ir[6]	in1
113	N/A	None	5.833 ns	ir[0]	jz
114	N/A	None	5.644 ns	ir[6]	mova

Tpd 图

结果分析及结论：

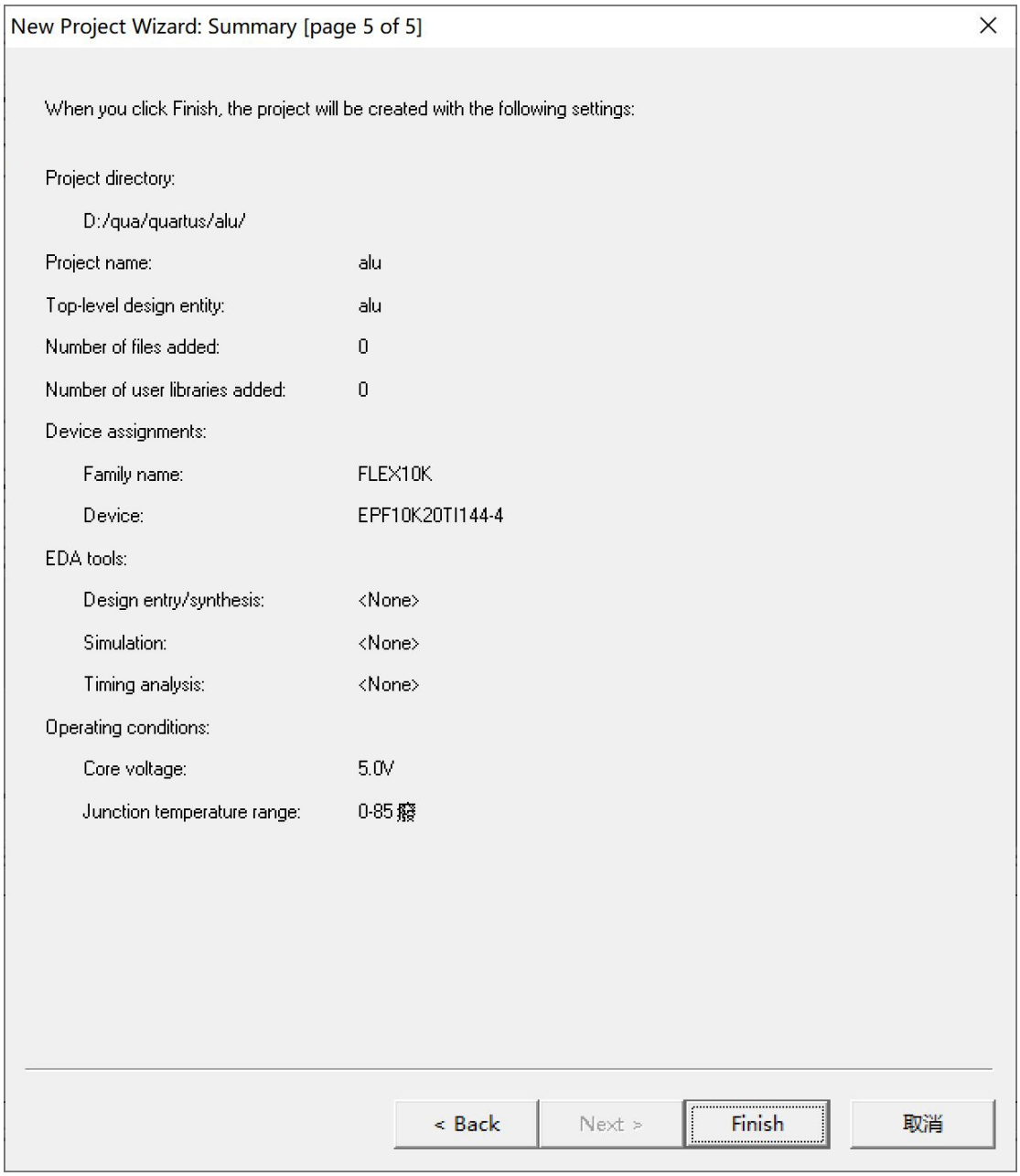
分析：由图可得，Timing Analyzer Summmary 总结所有经典定时分析的结果，并报告每个定时特性的最坏情况定时。比如从 ir[3]到 jmp 的最坏定时情况的 tpd 为 11.720ns。下面的 tpd 报告表则给出了源节点和目标节点之间的 tpd 延迟时间，比如第二行中 ir[5]到 add 的 tpd 为 11.699ns。

结论：实际连接图中个元器件连接之间是存在时间延迟的，而且不同的元器件之间的时间延迟也不相同。

PDF-XChange Editor Click to BUY NOW! www.tracker-software.com

2、算术逻辑单元 ALU

A）创建工程（选择的芯片为 family=FLEX10K；name=EPF10K20TI144-4）步骤同指令译码器



PDF-XChange Editor Click to BUY NOW! www.tracker-software.com

B）编写源代码

步骤同指令译码器

```
module alu(m,s,a,b,cf,zf,t);
  input wire m;
  input wire [3:0] s;
  input wire [7:0] a,b;
  output reg cf,zf;
  output reg [7:0] t;
  reg[8:0] temp;
  always @(m or s or a or b) begin
      if(m==1'b0&&s[3:0]==4'b1100) t=a;
      else if(m==1'b1) begin
          if(s[3:0]==4'b1001) begin
          t=a+b;
          if(t==8'b00000000) zf=1;
          else zf=0;

          temp=a+b;
          if(temp[8]==1'b0) cf=0;
          else cf=1;
        end
        else if(s[3:0]==4'b0110) begin
          t=b-a;
          if(t==8'b00000000) zf=1;
          else zf=0;

          temp=b-a;
          if(temp[8]==1'b0) cf=0;
          else cf=1;
        end
        else if(s[3:0]==4'b1011) t=a&b;
        else if(s[3:0]==4'b0101) t=~b;
        else if(s[3:0]==4'b1010||s[3:0]==4'b0100) t=b;
      end
 end

 endmodule
```

C）编译与调试（包含编译调试过程中的错误、警告信息以及资源消耗）

步骤同指令译码器

编译调试错误无

警告信息：

```
Warning (10240): Verilog HDL Always Construct warning at alu.v(8): inferring latch(es) for variable "t", which holds its previous value in one or more paths throu
Warning (10240): Verilog HDL Always Construct warning at alu.v(8): inferring latch(es) for variable "zf", which holds its previous value in one or more paths thr
Warning (10240): Verilog HDL Always Construct warning at alu.v(8): inferring latch(es) for variable "temp", which holds its previous value in one or more paths t
Warning (10240): Verilog HDL Always Construct warning at alu.v(8): inferring latch(es) for variable "cf", which holds its previous value in one or more paths thr
Warning: Latch cf$latch has unsafe behavior
Warning: Latch zf$latch has unsafe behavior
Warning: Latch t[0]$latch has unsafe behavior
Warning: Latch t[1]$latch has unsafe behavior
Warning: Latch t[2]$latch has unsafe behavior
Warning: Latch t[3]$latch has unsafe behavior
Warning: Latch t[4]$latch has unsafe behavior
Warning: Latch t[5]$latch has unsafe behavior
Warning: Latch t[6]$latch has unsafe behavior
Warning: Latch t[7]$latch has unsafe behavior
Warning: Ignored 7 CARRY_SUM primitives
Warning: Timing Analysis is analyzing one or more combinational loops as latches
Warning: Found pins functioning as undefined clocks and/or memory enables
Warning: Found 10 node(s) in clock paths which may be acting as ripple and/or gated clocks -- node(s) analyzed as buffer(s) resulting in clock skew
```

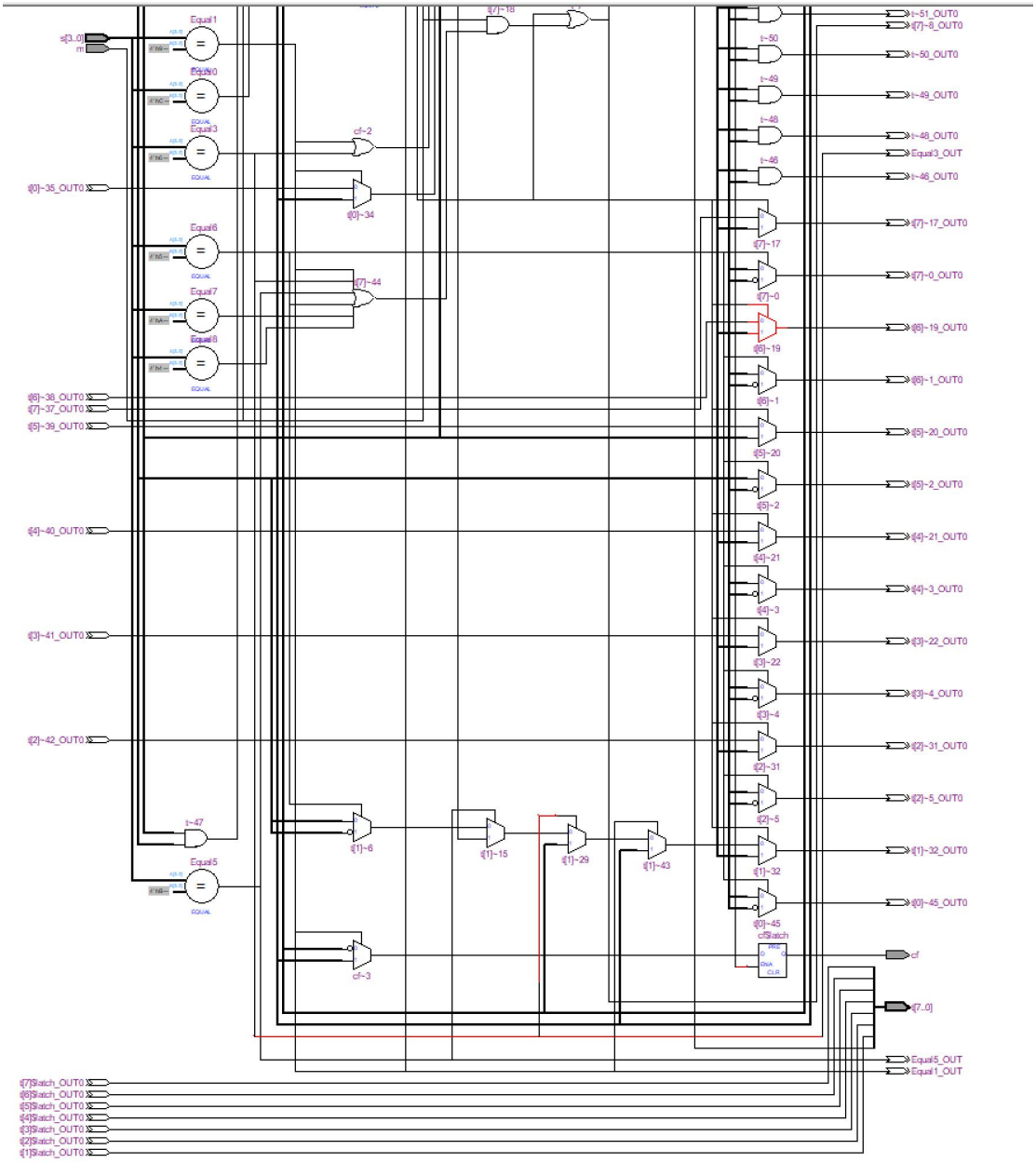
资源消耗：

Flow Summary

Flow Status	Successful - Mon Dec 06 21:00:58 2021
Quartus II Version	9.0 Build 235 06/17/2009 SP 2 SJ Web Edition
Revision Name	alu
Top-level Entity Name	alu
Family	FLEX10K
Device	EPF10K10TI144-4
Timing Models	Final
Met timing requirements	Yes
Total logic elements	83 / 576 (14 %)
Total pins	31 / 102 (30 %)
Total memory bits	0 / 6,144 (0 %)

PDF-XChange Editor
Click to BUY NOW!
www.tracker-software.com

D) RTL 视图



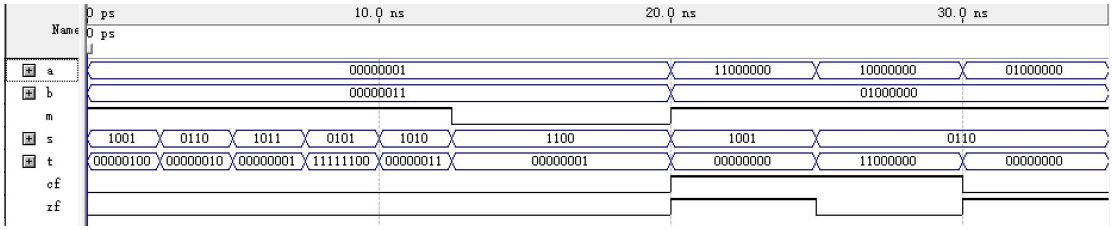
结果分析及结论：

分析：由视图可得，视图左边为输入，右边为输出。其中连接有一系列的元器件。比如比较器：当输入相等时输出 1，不相等时输出 0；还有大部分的 2-1 选择器构成，当控制信号为 0 时，输出第一位，控制信号为 1 时，输出第二位。图中输入信号为 m，s，a，b，输出信号为 t，cf，zf。各个输出端口之间通过导线相连。

结论：一个功能的实现需要经过多重门的处理后才能实现，一个元件的内部原理结构图十分复杂。

PDF-XChange Editor Click to BUY NOW! www.tracker-software.com

D) 功能仿真波形



功能仿真波形图

结果分析及结论：

分析：功能仿真是指不考虑器件延时和布线延时的理想情况下对源代码进行逻辑功能的验证。由仿真波形可得，对于输入状态的变化，输出结果实时变化，没有延迟，其结果与电路设计的真值表的结果相对应。

当控制信号 m 为 1，s 为 1001 时，执行 t=a+b

当控制信号 m 为 1，s 为 0110 时，执行 t=b-a

当控制信号 m 为 1，s 为 1011 时，执行 t=a&b

当控制信号 m 为 1，s 为 0101 时，执行 t=~b

当控制信号 m 为 1，s 为 1010 时，执行 t=b

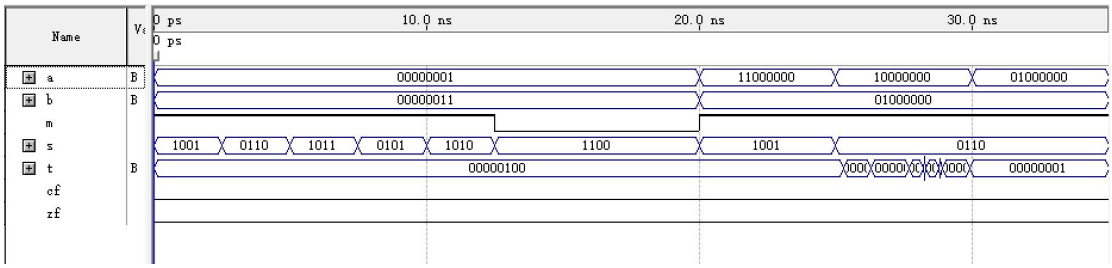
当控制信号 m 为 0，s 为 1100 时，执行 t=a

有进位和借位时 cf 为 1，否则为 0；

和为 0 或差为 0 时 zf 为 1，否则为 0；

结论：功能仿真操作简单，能体现和验证实验的功能，但忽略延迟的影响会使结果与实际结果有一定误差。

F) 时序仿真波形



结果分析及结论：

分析：时序仿真是指在布线后进行，是最接近真实器件运行的仿真，它与特定的器件有关，又包含了器件和布线的延时信息。由波形可得，当输入状态发生改变时，输出结果并未同时改变，而是有一定延迟，同时由于输入状态的改变，导致电路出现“冒险”，导致输出结果并未与预期结果相同。

结论：时序仿真可以用来验证程序在目标器件中的时序关系。同时考虑了器件的延迟后，其输出结果跟接近实际情况，但是考虑的情况过多，不容易操作，容易产生错误。时序仿真不仅反应出输出和输入的逻辑关系，同时还计算了时间的延时信息，是与实际系统更接近的一种仿真结果。不过，要注意的是，这个时间延时是仿真软件“估算”出来的。





G) 时序分析

Timing Analyzer Summary

	Type	Slack	Required Time	Actual Time	From	To	From Clock	To Clock	Failed Paths
1	Worst-case tsu	N/A	None	38.500 ns	a[0]	zf$latch	--	m	0
2	Worst-case tco	N/A	None	26.500 ns	t[6]$latch	t[6]	s[3]	--	0
3	Worst-case th	N/A	None	8.400 ns	a[6]	t[6]$latch	--	s[3]	0
4	Total number of failed paths								0

tpd

	Slack	Required P2P Time	Actual P2P Time	From	To
1	N/A	None	30.200 ns	s[0]	t[2]
2	N/A	None	29.600 ns	s[0]	t[0]
3	N/A	None	29.100 ns	a[0]	t[7]
4	N/A	None	29.100 ns	a[0]	zf
5	N/A	None	29.000 ns	s[0]	t[7]
6	N/A	None	29.000 ns	b[0]	t[7]
7	N/A	None	29.000 ns	b[0]	zf
8	N/A	None	28.800 ns	b[1]	t[7]
9	N/A	None	28.800 ns	b[1]	zf
10	N/A	None	28.500 ns	a[2]	t[7]
11	N/A	None	28.500 ns	a[2]	zf
12	N/A	None	28.400 ns	s[0]	t[6]
13	N/A	None	28.200 ns	b[3]	t[7]
14	N/A	None	28.200 ns	b[2]	t[7]
15	N/A	None	28.200 ns	s[0]	t[5]
16	N/A	None	28.200 ns	a[0]	t[5]
17	N/A	None	28.200 ns	a[0]	t[0]
18	N/A	None	28.200 ns	b[3]	zf
19	N/A	None	28.200 ns	b[2]	zf
20	N/A	None	28.100 ns	b[0]	t[5]
21	N/A	None	28.100 ns	b[0]	t[0]
22	N/A	None	28.000 ns	a[0]	t[6]
23	N/A	None	28.000 ns	s[2]	t[2]
24	N/A	None	28.000 ns	s[1]	t[2]
25	N/A	None	28.000 ns	s[3]	t[2]
26	N/A	None	27.900 ns	b[0]	t[6]
27	N/A	None	27.900 ns	b[1]	t[5]
28	N/A	None	27.700 ns	a[3]	t[7]
29	N/A	None	27.700 ns	b[1]	t[6]
30	N/A	None	27.700 ns	s[0]	t[3]
31	N/A	None	27.700 ns	a[3]	zf
32	N/A	None	27.600 ns	a[2]	t[5]
33	N/A	None	27.400 ns	a[2]	t[6]
34	N/A	None	27.400 ns	a[0]	t[2]
35	N/A	None	27.300 ns	b[3]	t[5]
36	N/A	None	27.300 ns	b[2]	t[5]
37	N/A	None	27.300 ns	b[0]	t[2]
38	N/A	None	27.200 ns	s[1]	t[0]
39	N/A	None	27.200 ns	s[3]	t[0]
40	N/A	None	27.100 ns	b[6]	t[7]
41	N/A	None	27.100 ns	b[3]	t[6]
42	N/A	None	27.100 ns	b[2]	t[6]
43	N/A	None	27.100 ns	b[6]	zf
44	N/A	None	27.000 ns	b[4]	t[7]
45	N/A	None	27.000 ns	b[1]	t[2]
46	N/A	None	27.000 ns	b[4]	zf
47	N/A	None	26.800 ns	s[2]	t[7]
48	N/A	None	26.800 ns	s[1]	t[7]
49	N/A	None	26.800 ns	s[3]	t[7]
50	N/A	None	26.800 ns	a[3]	t[5]
51	N/A	None	26.800 ns	a[2]	t[2]
52	N/A	None	26.700 ns	b[5]	t[7]
53	N/A	None	26.700 ns	a[1]	t[7]
54	N/A	None	26.700 ns	s[2]	t[0]
55	N/A	None	26.700 ns	b[5]	zf
56	N/A	None	26.700 ns	a[1]	zf
57	N/A	None	26.600 ns	b[7]	t[7]
58	N/A	None	26.600 ns	a[4]	t[7]
59	N/A	None	26.600 ns	a[3]	t[6]
60	N/A	None	26.600 ns	s[0]	t[4]
61	N/A	None	26.600 ns	s[0]	t[1]
62	N/A	None	26.600 ns	b[7]	zf
63	N/A	None	26.600 ns	a[4]	zf
64	N/A	None	26.500 ns	b[2]	t[2]
65	N/A	None	26.400 ns	a[6]	t[7]
66	N/A	None	26.400 ns	a[0]	t[1]
67	N/A	None	26.400 ns	a[6]	zf
68	N/A	None	26.300 ns	a[5]	t[7]
69	N/A	None	26.300 ns	b[0]	t[1]
70	N/A	None	26.300 ns	a[5]	zf
68	N/A	None	26.300 ns	a[5]	t[7]
69	N/A	None	26.300 ns	b[0]	t[1]
70	N/A	None	26.300 ns	a[5]	zf
71	N/A	None	26.200 ns	s[2]	t[6]
72	N/A	None	26.200 ns	s[1]	t[6]
76	N/A	None	26.100 ns	b[4]	t[5]
77	N/A	None	26.000 ns	s[2]	t[5]
78	N/A	None	26.000 ns	s[1]	t[5]
79	N/A	None	26.000 ns	s[3]	t[5]
80	N/A	None	26.000 ns	a[0]	t[4]
81	N/A	None	25.900 ns	b[4]	t[6]
82	N/A	None	25.900 ns	b[5]	t[5]
83	N/A	None	25.900 ns	b[0]	t[4]
84	N/A	None	25.800 ns	a[1]	t[5]
85	N/A	None	25.700 ns	a[4]	t[5]
86	N/A	None	25.700 ns	b[1]	t[4]
87	N/A	None	25.700 ns	a[0]	t[3]
88	N/A	None	25.600 ns	b[5]	t[6]
89	N/A	None	25.600 ns	a[1]	t[6]
90	N/A	None	25.600 ns	b[0]	t[3]
91	N/A	None	25.500 ns	a[4]	t[6]
92	N/A	None	25.500 ns	a[5]	t[5]
93	N/A	None	25.500 ns	s[2]	t[3]
94	N/A	None	25.500 ns	s[1]	t[3]
95	N/A	None	25.500 ns	s[3]	t[3]
96	N/A	None	25.400 ns	a[6]	t[6]
97	N/A	None	25.400 ns	a[2]	t[4]
98	N/A	None	25.400 ns	b[1]	t[3]
99	N/A	None	25.400 ns	s[0]	zf
100	N/A	None	25.200 ns	a[5]	t[6]
101	N/A	None	25.100 ns	b[3]	t[4]
102	N/A	None	25.100 ns	b[2]	t[4]
103	N/A	None	25.100 ns	a[2]	t[3]
104	N/A	None	25.000 ns	a[1]	t[2]
105	N/A	None	24.900 ns	b[3]	t[3]
106	N/A	None	24.800 ns	b[2]	t[3]
107	N/A	None	24.700 ns	s[0]	cf
108	N/A	None	24.600 ns	a[3]	t[4]
109	N/A	None	24.400 ns	s[2]	t[4]
110	N/A	None	24.400 ns	s[1]	t[4]
111	N/A	None	24.400 ns	s[3]	t[4]
112	N/A	None	24.400 ns	a[3]	t[3]
113	N/A	None	24.300 ns	a[7]	t[7]
114	N/A	None	24.300 ns	s[2]	t[1]
115	N/A	None	24.300 ns	s[1]	t[1]
116	N/A	None	24.300 ns	s[3]	t[1]
117	N/A	None	24.300 ns	a[7]	zf
118	N/A	None	24.100 ns	a[1]	t[1]
119	N/A	None	24.000 ns	b[4]	t[4]
120	N/A	None	23.600 ns	a[4]	t[4]
121	N/A	None	23.600 ns	a[1]	t[4]
122	N/A	None	23.500 ns	s[2]	zf
123	N/A	None	23.500 ns	s[1]	zf
124	N/A	None	23.500 ns	s[3]	zf
125	N/A	None	23.400 ns	a[0]	cf
126	N/A	None	23.300 ns	a[1]	t[3]
127	N/A	None	23.300 ns	b[0]	cf
128	N/A	None	23.100 ns	b[1]	cf
129	N/A	None	22.800 ns	s[2]	cf
130	N/A	None	22.800 ns	s[1]	cf
131	N/A	None	22.800 ns	s[3]	cf
132	N/A	None	22.800 ns	a[2]	cf
133	N/A	None	22.500 ns	m	t[0]
134	N/A	None	22.500 ns	b[3]	cf
135	N/A	None	22.500 ns	b[2]	cf
136	N/A	None	22.000 ns	a[3]	cf
137	N/A	None	21.400 ns	b[6]	cf
138	N/A	None	21.300 ns	b[4]	cf
139	N/A	None	21.000 ns	b[5]	cf
140	N/A	None	21.000 ns	a[1]	cf
138	N/A	None	21.300 ns	b[4]	cf
139	N/A	None	21.000 ns	b[5]	cf
140	N/A	None	21.000 ns	a[1]	cf
141	N/A	None	20.900 ns	a[4]	cf
142	N/A	None	20.800 ns	b[7]	cf
143	N/A	None	20.700 ns	a[6]	cf
144	N/A	None	20.600 ns	a[5]	cf
145	N/A	None	19.000 ns	m	zf
146	N/A	None	18.800 ns	m	cf
147	N/A	None	18.500 ns	a[7]	cf
148	N/A	None	15.400 ns	m	t[6]
149	N/A	None	15.100 ns	m	t[2]
150	N/A	None	14.700 ns	m	t[5]
151	N/A	None	14.700 ns	m	t[3]
152	N/A	None	14.300 ns	m	t[7]
153	N/A	None	13.700 ns	m	t[1]
154	N/A	None	13.100 ns	m	t[4]





结果分析及结论：

分析：由图可得，Timing Analyzer Summmary 总结所有经典定时分析的结果，并报告每个定时特性的最坏情况定时。比如从 a[0]到 zf 的最坏定时情况的 tpd 为 38.50ns。下面的 tpd 报告表则给出了源节点和目标节点之间的 tpd 延迟时间，比如第二行中 s[0]到 t[0]的 tpd 为 29.6ns。

结论：实际连接图中个元器件连接之间是存在时间延迟的，而且不同的元器件之间的时间延迟也不相同。

四、思考题

1. 指令译码器必须要 16 个输出吗？可否将一些输出合并，哪些可以合并，为什么？

答：不一定。jmp 和 add 可以合并起来，因为 jmp 是将 add 后的结果写入 pc 中，则可以进行 add 操作后直接进行写入操作。Add 和 sub 和 and 操作可以合并，因为这三个操作类似，且输出为使能信号，故可以用一个合并使能信号来作为三个输出的共同使能信号。

2. ALU 中的 S[3..0]控制信号是来自哪里或者说与什么信息相同？

答：来自指令码 ir 的前四位。

3、为何 S[3..0]等于 1100 时将输入 a 传给 t，S[3..0]等于 1010 或 0100 时将输入 b 传给 t？

S[3..0]为控制信号，当 S[3..0]输入为 1100 时，控制输出 t 等于 a，S[3..0]等于 1010 或 0100 时，t 等于 b，此时 alu 相当于选择器。

五、实验总结、必得体会及建议

1、从需要掌握的理论、遇到的困难、解决的办法以及经验教训等方面进行总结。

基本了解了简易模型机的内部结构和工作原理。同时熟悉了译码器、ALU 的工作原理。学会使用 Verilog 语言编写电路。但遇到了一些困难，比如不懂使用 QuartusII 进行波形仿真。但通过上网查询相关资料和询问同学后得以解决问题，并通过分析报告发现电路中的问题。还有不懂根据实验要求的功能表输出相对应的结果，不懂使用 always 和 assign 语句等，通过上网查询资料等都得以解决。以后再遇到相关问题时，可以通过先上网查询资料，再询问老师和同学来解决。

2、对本实验内容、过程和方法的改进建议（可选项）。

希望在元件的功能表中能给出当输入除了已给出的输出情况外的输入，输出结果应该取什么。